数据结构与算法 · 第二章 线性表的结构分析与应用

> 主教材：李刚等《数据结构（C语言版）》微课第2版 · 人邮 | 参考：严蔚敏《数据结构》清华

学习目标

- 掌握线性表的逻辑结构及基本运算
- 掌握顺序表的插入、删除等基本运算的实现算法
- 掌握单链表的插入、删除等基本运算的实现算法
- 掌握顺序表和单链表各自的特点及适用场合

第一部分 · 线性表的逻辑结构与基本运算

2.1 什么是线性表？

大白话： 线性表就是**排成一队的一串数据**——像食堂打饭的队伍，每个人前面恰好一个人、后面恰好一个人，秩序井然。

形式化定义：

线性表是由 n（n ≥ 0）个**相同类型**的数据元素组成的**有限序列**，记作：

$$(a_1, a_2, a_3, \ldots, a_{i-1}, a_i, a_{i+1}, \ldots, a_n)$$

几个关键点：

- **有限**：元素个数 n 是有限的，n = 0 时称为**空表**。
- **相同类型**：一个线性表里所有元素的数据类型必须一致。
- **有序**：元素之间有先后次序，位置不能随意打乱。
- a_1 称为**表头元素**，a_n 称为**表尾元素**。
- 除 a_1 外，每个元素**有且仅有一个直接前驱**；除 a_n 外，每个元素**有且仅有一个直接后继**。

> **一句话记住：线性表 = 一对一（前驱后继唯一）的线性结构。**

这正是第一章 1.4 节讲过的四种逻辑结构中的**线性结构**。回顾一下：集合（点名册）、**线性（食堂排队）**、树形（校长→主任→老师）、图形（微信好友网）。

生活中的线性表举例：

场景	元素是什么	关系
食堂排队的队伍	每个人	前后有序，一个接一个
班级花名册	每名学生	按学号从小到大排列
手机通讯录	每个联系人	一个挨着一个排列
试卷上的题目	每道题	第1题→第2题→…→第30题

场景	元素是什么	关系
学生成绩表	每条成绩记录	按录入顺序排列

2.2 线性表的基本运算（操作）

线性表不只是一堆数据，还必须能"用"——所以要在它上面定义一组基本操作：

运算	写法（示例）	说明
初始化	`InitList(&L)`	造一个空表出来
求表长	`ListLength(L)`	表里有几个元素？
判空	`ListEmpty(L)`	表是不是空的？
按位查找	`GetElem(L, i, &e)`	把第 i 个元素取出来
按值查找	`LocateElem(L, e)`	值等于 e 的元素在第几位？
插入	`ListInsert(&L, i, e)`	在第 i 个位置插一个新元素
删除	`ListDelete(&L, i, &e)`	删掉第 i 个元素，并把它带回来
遍历	`PrintList(L)`	从头到尾走一遍，逐个输出
清空	`ClearList(&L)`	把表里的元素全清掉

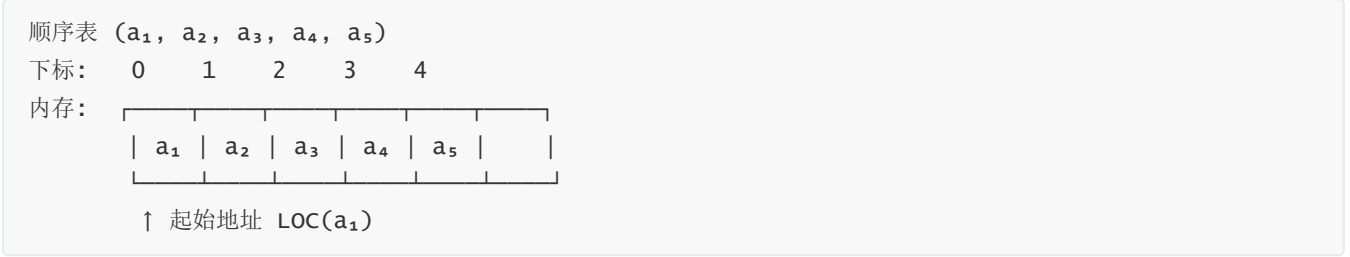
> **注意：** 凡是会"改变线性表本身"的运算（初始化、插入、删除、清空），参数都用**指针**（如 `&L` 或 `SqList *L`）；只"读表"的运算（求长、查找、遍历）传值即可。这在第一章第二讲的登录模块案例里已经用过同样的套路。

第二部分 · 顺序表（顺序存储结构）

2.3 顺序表长什么样？

存储思想： 用一组**连续的存储单元**依次存放线性表的元素——逻辑上相邻的两个元素，物理地址上也相邻。

大白话： 就像**电影院的连座票**，一排座位紧挨着，第 1 个人坐 1 号位、第 2 个人坐 2 号位……想找第 i 个人，直接走到第 i 个座位就行。



关键公式（随机访问的基础）：

```
LOC(aᵢ) = LOC(a₁) + (i - 1) × L          （L 为每个元素占用的字节数）
```

因为地址可以直接算出来，所以顺序表**按位查找是 O(1)** —— 一步到位。

类型定义（C语言）：

```c
#define MAXSIZE 100         // 顺序表的最大容量

typedef struct {
    ElemType data[MAXSIZE]; // 用数组存储数据元素
    int      length;        // 顺序表的当前长度
} SqList;
```

要点：

- `data` 是数组，负责**存**元素；`length` 是整型，负责**记**当前有几个元素。
- 由于数组长度必须事先确定，顺序表属于**静态分配**（若用 `malloc` 动态申请，则称为动态分配）。
- 表满（`length == MAXSIZE`）时不能再插，必须判满。

2.4 顺序表的插入算法（重点）

任务： 在顺序表 L 的第 i 个位置（1 ≤ i ≤ length+1）插入一个新元素 e。

思路演示： 假设表里已有 `{10, 20, 30, 40}`，要在第 2 个位置插入 `15`：

```
插入前：  ┌────┬────┬────┬────┬────┐
          | 10 | 20 | 30 | 40 |    |
          └────┴────┴────┴────┴────┘
第1步：腾位置——从最后一个元素开始，依次往后挪一格（必须"从后往前"！）
          ┌────┬────┬────┬────┬────┐
          | 10 |    | 20 | 30 | 40 |
          └────┴────┴────┴────┴────┘
第2步：填空——把 15 放进第 2 个位置
          ┌────┬────┬────┬────┬────┐
          | 10 | 15 | 20 | 30 | 40 |
          └────┴────┴────┴────┴────┘
```

为什么必须"从后往前"移动？ 因为如果从前往后挪，a_i 会把 a_{i+1} 覆盖掉，后面的数据就被"吃掉"了——就像坐满人的一排座位，你要在中间插一个人，必须**从最后一个人开始往外挪。**

算法实现：

```c
// 在顺序表 L 的第 i 个位置插入元素 e，成功返回 1，失败返回 0
int ListInsert(SqList *L, int i, ElemType e) {
    int j;
    // ① 判插入位置是否合法：i 的范围是 1 ~ length+1
    if (i < 1 || i > L->length + 1) {
        return 0;
    }
    // ② 判表是否已满
```

```
    if (L->length >= MAXSIZE) {
        return 0;
    }
    // ③ 腾位置：从最后一个元素开始，依次后移一格
    for (j = L->length - 1; j >= i - 1; j--) {
        L->data[j + 1] = L->data[j];
    }
    // ④ 填空：把新元素 e 放到第 i 个位置（下标 i-1）
    L->data[i - 1] = e;
    // ⑤ 表长加 1
    L->length++;
    return 1;
}
```

移动次数与时间复杂度分析：

情况	插入位置	需要移动的元素个数
最好	表尾（i = n+1）	0 个
最坏	表头（i = 1）	n 个
平均	等概率插入	n / 2 个

> **结论：顺序表插入的时间复杂度为 O(n)**，主要开销都在"搬数据"上；表越长，插入越费劲。

2.5 顺序表的删除算法（重点）

任务： 删除顺序表 L 的第 i 个元素（1 ≤ i ≤ length），并用 `*e` 把被删的元素带回来。

思路演示： 表里是 `{10, 15, 20, 30, 40}`，删除第 2 个元素 `15`：

```
删除前：  ┌────┬────┬────┬────┬────┐
          | 10 | 15 | 20 | 30 | 40 |
          └────┴────┴────┴────┴────┘
第1步：保存要删的元素 15
第2步：填补空缺——从被删位置的下一个元素开始，依次往前挪一格（"从前往后"）
          ┌────┬────┬────┬────┬────┐
          | 10 | 20 | 30 | 40 |    |
          └────┴────┴────┴────┴────┘
第3步：表长减 1
```

算法实现：

```
// 删除顺序表 L 的第 i 个元素，被删元素用 *e 带回，成功返回 1，失败返回 0
int ListDelete(SqList *L, int i, ElemType *e) {
    int j;
    // ① 判删除位置是否合法：i 的范围是 1 ~ length
    if (i < 1 || i > L->length) {
        return 0;
    }
```

```
    // ② 先把被删元素保存起来
    *e = L->data[i - 1];
    // ③ 填补空缺：从前往后依次前移一格
    for (j = i; j < L->length; j++) {
        L->data[j - 1] = L->data[j];
    }
    // ④ 表长减 1
    L->length--;
    return 1;
}
```

移动次数与时间复杂度分析：

情况	删除位置	需要移动的元素个数
最好	表尾（i = n）	0 个
最坏	表头（i = 1）	n - 1 个
平均	等概率删除	(n - 1) / 2 个

> **结论：顺序表删除的时间复杂度也是 O(n)**，同样耗在"搬数据"上。

插入 vs 删除 一句话对比： 插入是"**从后往前挪，腾出空位**"；删除是"**从前往后挪，填上坑**"。

2.6 顺序表的查找

按位查找（随机访问）：

```
// 取第 i 个元素，用 *e 带回
int GetElem(SqList L, int i, ElemType *e) {
    if (i < 1 || i > L.length) {
        return 0;
    }
    *e = L.data[i - 1];    // 下标 = 位序 - 1，一步到位
    return 1;
}
```

> 顺序表**按位查找的时间复杂度是 O(1)**：地址可以直接用公式算出来，不需要从前往后找。

按值查找（顺序查找）：

```c
// 查找值为 e 的元素，返回其位序（找不到返回 0）
int LocateElem(SqList L, ElemType e) {
    int i;
    for (i = 0; i < L.length; i++) {
        if (L.data[i] == e) {   // 结构体比较需逐个数据项比较
            return i + 1;
        }
    }
    return 0;
}
```

> 按值查找要**一个一个比对**，时间复杂度为 **O(n)**——和第一章练习中"在没排序的书架上找一本书"是同一个道理。

2.7 顺序表的优缺点

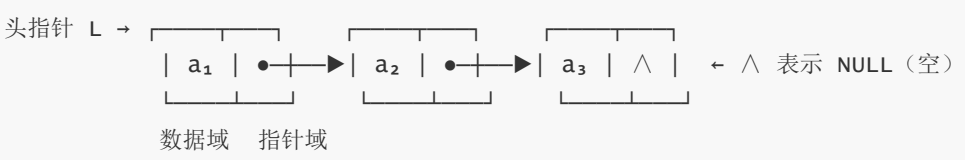
方面	顺序表
优点 1	可以**随机访问**（按位查找 O(1)），想取第几个就取第几个
优点 2	存储密度高，不用额外空间存指针，只存数据本身
缺点 1	插入、删除要**大量移动元素**（O(n)），效率低
缺点 2	需要**预先分配固定容量**，估计大了浪费、小了会溢出
缺点 3	表长变化大时，扩容易造成内存碎片或需要整体搬迁

第三部分 · 单链表（链式存储结构）

2.8 单链表长什么样？

存储思想： 每个元素单独放在一个**结点**里，结点除了存数据，还存一个**指向下一个结点的指针**。各结点在内存里可以**不连续**。

大白话： 就像**寻宝游戏**——每一站只告诉你"下一站在哪儿"。你手里只要有第一站的地址，就能一站一站走到底。

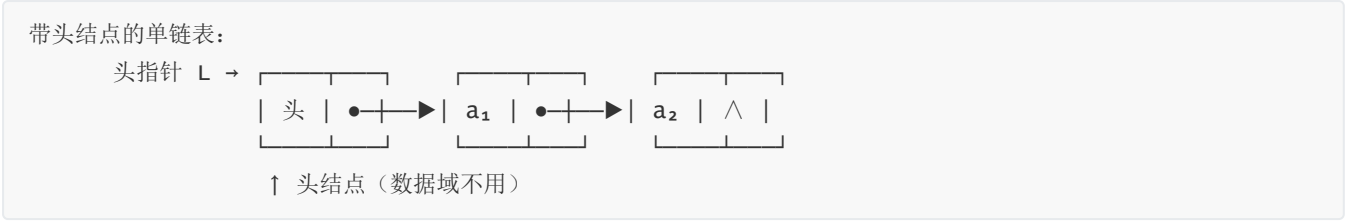


类型定义（C语言）：

```c
typedef struct LNode {
    ElemType      data;    // 数据域：存放数据元素
    struct LNode *next;    // 指针域：存下一个结点的地址
} LNode, *LinkList;        // LNode 是结点类型，LinkList 是指向结点的指针类型
```

两个重要概念：

- **头指针 L：** 指向链表第一个结点的指针，是链表的"总入口"，丢了它整条链表就找不到了。
- **头结点：** 在第一个"真"元素结点之前设一个**不存数据**的结点。它的作用是统一操作、简化插入删除的判断（插入和删除第一个位置时不用特殊处理）。



顺序表 vs 单链表的"第一印象"差别：

对比点	顺序表	单链表
存储位置	一段连续空间	结点分散在内存各处
找第 i 个元素	算地址，O(1)	从头数过去，O(n)
插入/删除	要挪一堆元素	只改几个指针

2.9 单链表的插入算法（重点）

任务： 在第 i 个位置插入值为 e 的新结点。

关键思路： 单链表只能"从前往后"走，所以要插入到第 i 个位置，就必须**先找到第 i-1 个结点**（前驱结点），再在它后面"接"上新结点。

指针修改顺序（超级重要）：

```
s->next = p->next;    // ① 先让新结点接上后面的结点
p->next = s;          // ② 再让前驱结点指向新结点
```

能不能把两句颠倒？绝对不能！

```
错误顺序 p->next = s; 后，原来的链断了：

正确顺序：  p → s → (原来的后继)      ✅ 两条链都还在
错误顺序：  p → s XXX                 ❌ p->next 被覆盖，后面整串结点全丢了
```

> **口诀：先接后、再断前**（先让新结点指向后继，再让前驱指向新结点）。

算法实现（带头结点）：

```
// 在第 i 个位置插入元素 e，成功返回 1，失败返回 0
int ListInsert(LinkList L, int i, ElemType e) {
    LNode *p, *s;
    // ① 找到第 i-1 个结点（前驱），i=1 时 p 就是头结点
    p = GetElem(L, i - 1);
```

```
    if (p == NULL) {
        return 0;                          // 插入位置不合法
    }
    // ② 申请新结点
    s = (LNode *)malloc(sizeof(LNode));
    if (s == NULL) {
        return 0;                          // 内存申请失败
    }
    s->data = e;
    // ③ 关键两步：先接后、再断前
    s->next = p->next;
    p->next = s;
    return 1;
}
```

> **时间复杂度：** 若已给出前驱结点 p，插入本身只需 O(1)；否则要先找前驱，整体为 **O(n)**（查找的开销）。

小技巧：前插可以"变"后插

如果只给了某个结点 p，要在 p **前面**插入 s，可以先"偷梁换柱"：把 s 插到 p 后面，然后交换两者的数据域——这样就不用找前驱了，复杂度 O(1)。

2.10 单链表的删除算法（重点）

任务： 删除第 i 个结点，并把它的数据带回来。

为什么必须找前驱？ 因为要删除结点 q，必须让 q 的前一个结点的 `next` **绕过 q** 直接指向 q 的后继，否则链表会从 q 处断开。

```
删除前:  p → q → r
删除后:  p ———→ r          （q 变成"孤儿"，再 free 掉）
```

算法实现：

```
// 删除第 i 个结点，数据用 *e 带回，成功返回 1，失败返回 0
int ListDelete(LinkList L, int i, ElemType *e) {
    LNode *p, *q;
    // ① 找到第 i-1 个结点（前驱）
    p = GetElem(L, i - 1);
    if (p == NULL || p->next == NULL) {
        return 0;                          // 前驱不存在，或第 i 个结点不存在
    }
    // ② q 指向要删除的结点
    q = p->next;
    // ③ 把数据保存出来
    *e = q->data;
    // ④ 跳过 q：让前驱直接指向 q 的后继
    p->next = q->next;
    // ⑤ 释放被删结点的内存（关键！否则内存泄漏）
    free(q);
    return 1;
```

```
}
```

> **两个易错点：** ① 删除后一定要 `free(q)`，否则内存泄漏（顺序表因为是一整块数组，不存在这个操作）；② 如果删除的是**尾结点**，`q->next` 是 NULL，`p->next = q->next` 恰好把尾结点的 next 置空，逻辑依然成立。
>
> **时间复杂度：** 主要开销在找前驱，为 **O(n)**。

2.11 单链表的查找

按位查找（从头数过去）：

```
// 返回第 i 个结点的指针；i=0 时返回头结点
LNode *GetElem(LinkList L, int i) {
    int j = 1;
    LNode *p = L->next;     // 从头结点的下一个结点（第一个真元素）开始
    if (i < 0) {
        return NULL;
    }
    if (i == 0) {
        return L;
    }
    while (p != NULL && j < i) {    // 顺着 next 一个一个往后走
        p = p->next;
        j++;
    }
    return p;                       // 走到第 i 个就返回，走到 NULL 说明 i 超范围
}
```

> **时间复杂度 O(n)**：单链表没有“随机访问”能力，找第 i 个只能从头数——这正是第一章说的“在没排序的书架上找书”。

按值查找：

```
// 查找值为 e 的结点，返回其指针
LNode *LocateElem(LinkList L, ElemType e) {
    LNode *p = L->next;
    while (p != NULL && p->data != e) {
        p = p->next;
    }
    return p;     // 找到返回结点指针，找不到返回 NULL
}
```

2.12 单链表的建立：头插法与尾插法

头插法（新结点总插在表头）——结果是倒序：

```c
void CreateListHead(LinkList L, int n) {
    LNode *s;
    int i;
    L->next = NULL;                      // 先建立空表（只有头结点）
    for (i = 0; i < n; i++) {
        s = (LNode *)malloc(sizeof(LNode));
        scanf("%d", &s->data);
        s->next = L->next;               // 新结点的 next 指向原来的第一结点
        L->next = s;                     // 头结点指向新结点
    }
}
```

尾插法（新结点总接在表尾）——结果是正序：

```c
void CreateListTail(LinkList L, int n) {
    LNode *s, *r = L;                    // r 是尾指针，初始指向头结点
    int i;
    L->next = NULL;
    for (i = 0; i < n; i++) {
        s = (LNode *)malloc(sizeof(LNode));
        scanf("%d", &s->data);
        r->next = s;                     // 新结点接到表尾
        r = s;                           // 尾指针后移
    }
    r->next = NULL;                      // 表尾结点的 next 置空
}
```

> **对比记忆：** 头插法不用找表尾，代码短、效率一样是 O(n)，但输出顺序会**反过来**；要按输入顺序输出就用尾插法。

2.13 单链表的优缺点

方面	单链表
优点 1	插入、删除**不用移动元素**，只改指针，效率高
优点 2	**动态分配**内存，表长变化灵活，不需要预留容量
缺点 1	不能随机访问，按位查找要 **O(n)**
缺点 2	每个结点多花一个指针的空间，**存储密度低**
缺点 3	指针操作繁琐，容易出现断链、内存泄漏等问题

第四部分 · 两种存储结构怎么选

2.14 顺序表 vs 单链表 对比表（重点）

比较维度	顺序表（顺序存储）	单链表（链式存储）
存储空间	连续的一段空间	结点可分散，用指针相连
存储密度	高（只存数据）	低（数据 + 指针）
表长	需预先固定（静态分配）	动态申请，灵活增减
按位查找	**O(1)**（随机访问）	**O(n)**（从头数）
按值查找	O(n)	O(n)
插入元素	**O(n)**（要挪元素，平均 n/2 个）	**O(1)**（已知前驱时，只改指针）
删除元素	**O(n)**（要挪元素，平均 (n-1)/2 个）	**O(1)**（已知前驱时，只改指针）
是否要移动元素	要	不要
额外开销	可能浪费预留空间	每个结点多一个指针域
编程难度	简单	指针操作较难，易出错

> **一句话记住：顺序表“查得快、改得慢”，链表“改得快、查得慢”。**

2.15 适用场合（选型指南）

选顺序表，当：

- 表长**可以预估**，且变化不大（如一个班级最多 50 人）。
- 主要操作是**按位置访问 / 频繁查找**（如按学号取某位学生的成绩）。
- **很少插入、删除**，或只在表尾增删。
- 要求**节省存储空间**（存储密度高）。

> 例：学生成绩表、班级花名册、题库按题号访问、教学评价系统的“用户表”。

选单链表，当：

- 表长**无法预估**，需要动态增长（如系统里不断新增的答题记录）。
- **频繁在中间插入、删除**（如按成绩排名插队、删除已注销用户）。
- 不需要随机访问，主要靠**遍历**处理数据。

> 例：教师名单动态维护、答题记录链、消息/通知队列、错题本动态收集。

再进一步（呼应第一章的复杂度思想）：

系统功能	推荐结构	理由
学生成绩表（按学号查分）	**顺序表**	按位查找 O(1)，查分最频繁

系统功能	推荐结构	理由
教师/用户名单（随时增删）	**单链表**	插入删除 O(1)，表长不定
试卷题目列表	**顺序表**	题号就是位序，顺序访问为主
待批改的答卷队列	**单链表**（配合队列）	先进先出，动态到达
系统操作日志	**顺序表 / 链表均可**	只追加、顺序读取

第五部分 · 本章小结 & 想一想

2.16 本章小结

主题	核心结论
线性表的逻辑结构	一对一，除首尾外每个元素有且仅有一个前驱和一个后继
基本运算	初始化、求长、判空、按位/按值查找、插入、删除、遍历、清空
顺序表	数组 + length；按位查找 O(1)；插入 O(n)、删除 O(n)（要挪元素）
单链表	结点（数据域 + 指针域）+ 头结点；插入、删除只改指针，但找前驱要 O(n)
选型	查得多、长固定 → 顺序表；增删频繁、长不定 → 单链表

本章知识地图：

```
线性表（逻辑结构：一对一）
├── 顺序存储 → 顺序表 SqList { data[]; length; }
|       ├── 按位查找 O(1)       ← 地址公式 LOC(aᵢ)=LOC(a₁)+(i-1)×L
|       ├── 插入 O(n)           ← 从后往前挪，腾空位
|       └── 删除 O(n)           ← 从前往后挪，填上坑
└── 链式存储 → 单链表 LNode { data; *next; }
        ├── 按位查找 O(n)       ← 从头数过去
        ├── 插入 O(1)           ← 先接后、再断前
        ├── 删除 O(1)           ← 找前驱、跳过去、free 掉
        └── 建表：头插法（倒序）/ 尾插法（正序）
```

2.17 想一想

① 顺序表插入元素时，为什么必须“从后往前”移动？
② 单链表的 `s->next = p->next;` 和 `p->next = s;` 能不能交换顺序？
③ 单链表删除结点时，为什么一定要先找到前驱结点？
④ 一个“学生成绩表”和一个“随时变化的教师名单”，分别该用哪种结构？

2.17.1 顺序表插入时，为什么必须“从后往前”移动？

大白话回答：

> 想象一排坐满人的连座，你要在第 2 个位置挤进一个人。你必须让**最后一个人先往外挪一格**，然后倒数第二个挪、……一路挪到第 2 个人。如果反着来（从第 2 个人开始往后挪），第 2 个人会直接把第 3 个人“顶掉”，数据就丢了。

技术说法： 元素后移的目标位置是 `j+1`，如果从前往后执行，`data[j]` 会先把尚未搬走的 `data[j+1]` 覆盖掉，导致数据丢失。循环条件必须写成 `for (j = L->length - 1; j >= i - 1; j--)`。

2.17.2 单链表的两句指针赋值能不能交换顺序？

答案：不能。

```
s->next = p->next;    // ① 先让新结点接住后面那一串
p->next = s;          // ② 再让前驱指向新结点
```

为什么？ 如果先执行 `p->next = s;`，那么 p 原本记录的“后继地址”就被覆盖了，`s->next` 再也拿不到后面的结点——**从 s 往后的整条链就全丢了**（内存泄漏 + 断链）。

> **口诀：先接后、再断前。**

2.17.3 单链表删除时，为什么一定要先找到前驱？

回答： 因为单链表是**单向**的，每个结点只知道“下一个是谁”，不知道“上一个是谁”。

要让目标结点 q 从链上消失，唯一的办法是让**它的前一个结点的 next 直接指向 q 的后继**：

```
p → q → r    →    p ──→ r      （再把 q free 掉）
```

所以必须先用 `GetElem(L, i-1)` 找到前驱 p。这也是单链表删除要先 O(n) 查找的原因。（如果是双向链表，就能直接由 q 找到前驱，操作会方便一些。）

2.17.4 学生成绩表 vs 动态教师名单，分别用什么结构？

场景	推荐结构	理由
学生成绩表（主要是按学号查分）	**顺序表**	按位/按学号定位快（O(1)），查询最频繁；人数固定，插入删除少
随时变化的教师名单（不断入职、离职）	**单链表**	插入、删除只改指针（O(1)），人数不定，不需要预留容量

一句话选型口诀：

> **“查得多、长固定”——用顺序表；**
> **“改得多、长不定”——用单链表。**

【额外小彩蛋 —— 方便你记忆】

> 线性表是排排队，前驱后继一个个；
> 顺序表住连座房，查得快来搬得忙；
> 单链表玩寻宝图，改得快来找得苦；
> 先接后再断前，选对结构省一半！